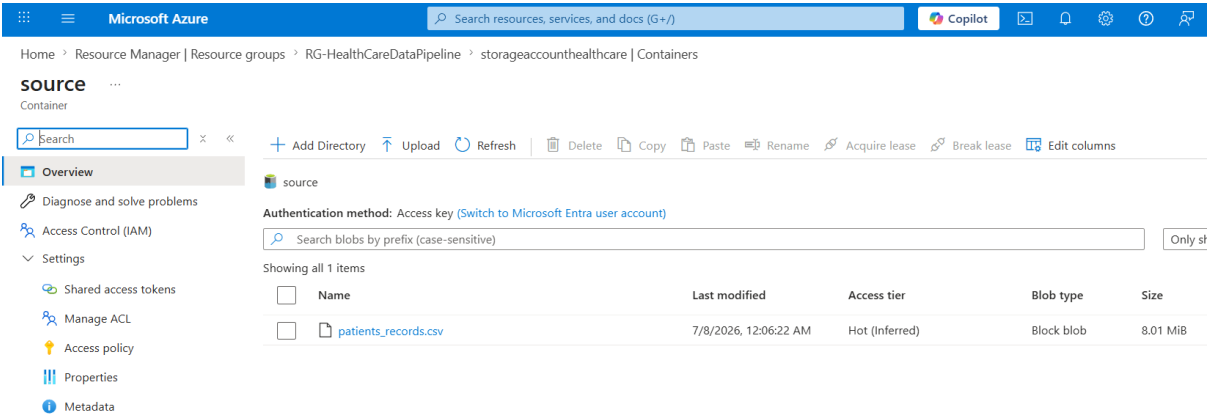


Healthcare Data Pipeline Project Screenshots

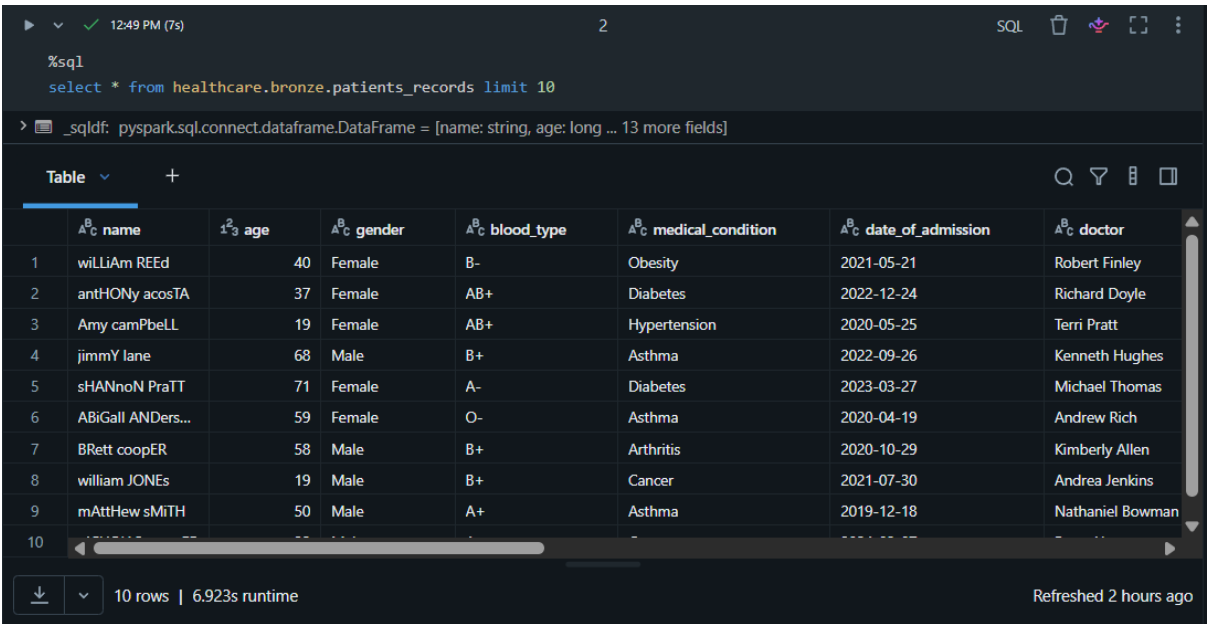
Screenshot 1: Source Data in Azure Data Lake Storage Gen2



Description:

The healthcare patient dataset is stored in Azure Data Lake Storage Gen2 inside the source container. This serves as the raw data source for the data pipeline before ingestion into Databricks.

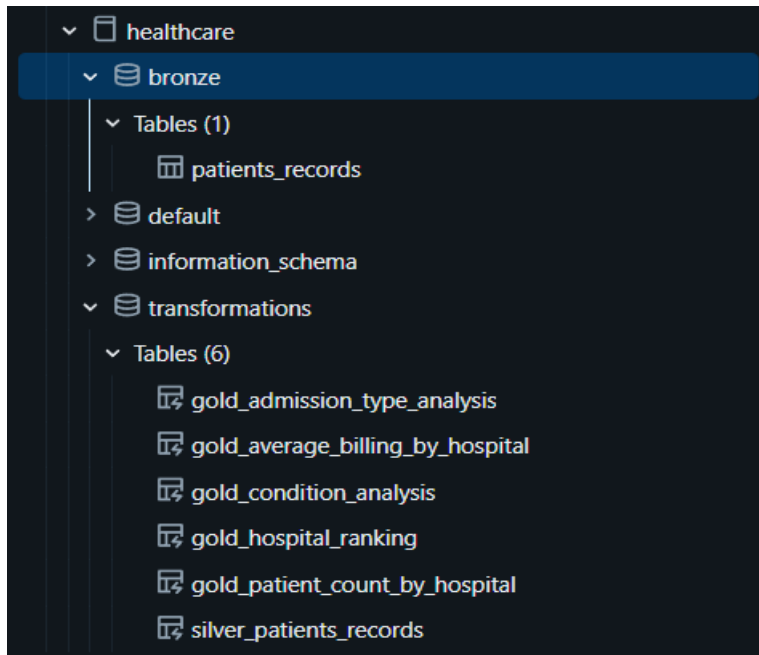
Screenshot 2: Bronze Layer



Description:

The raw healthcare dataset has been successfully ingested into the Bronze layer using Delta Lake without applying any transformations. This layer preserves the original data for traceability and future reprocessing.

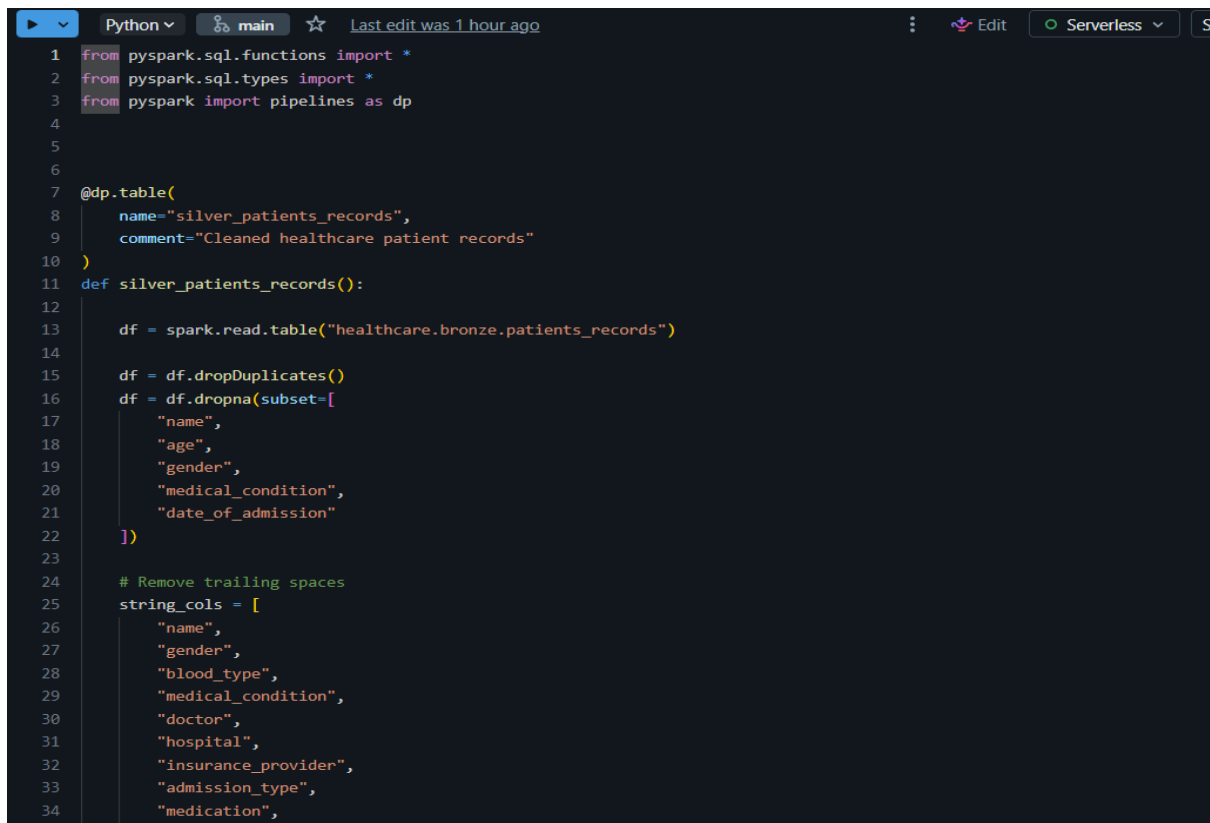
Screenshot 3: Pipeline Folder Structure



Description:

The Databricks Lakeflow Pipeline is organized into separate Silver and Gold transformation modules, ensuring a modular and maintainable pipeline structure.

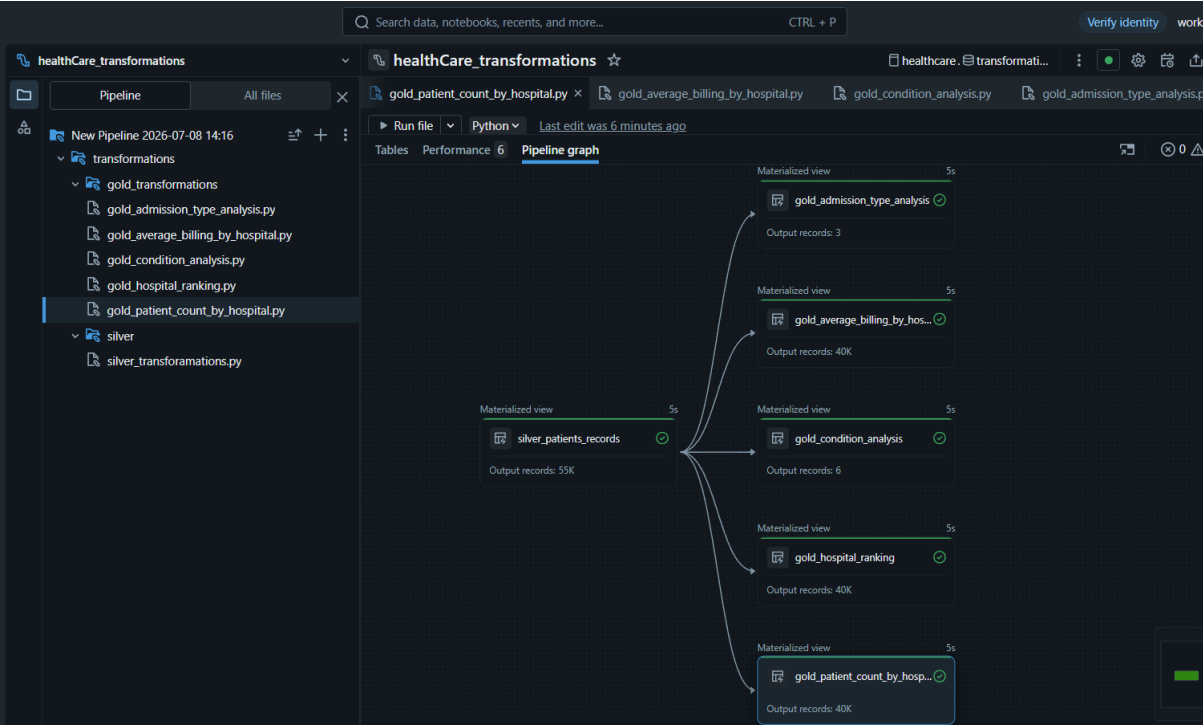
Screenshot 4: Silver Transformation



Description:

The Silver layer performs data cleansing, duplicate removal, null handling, data type conversion, and standardization to prepare clean and reliable datasets for downstream analytics.

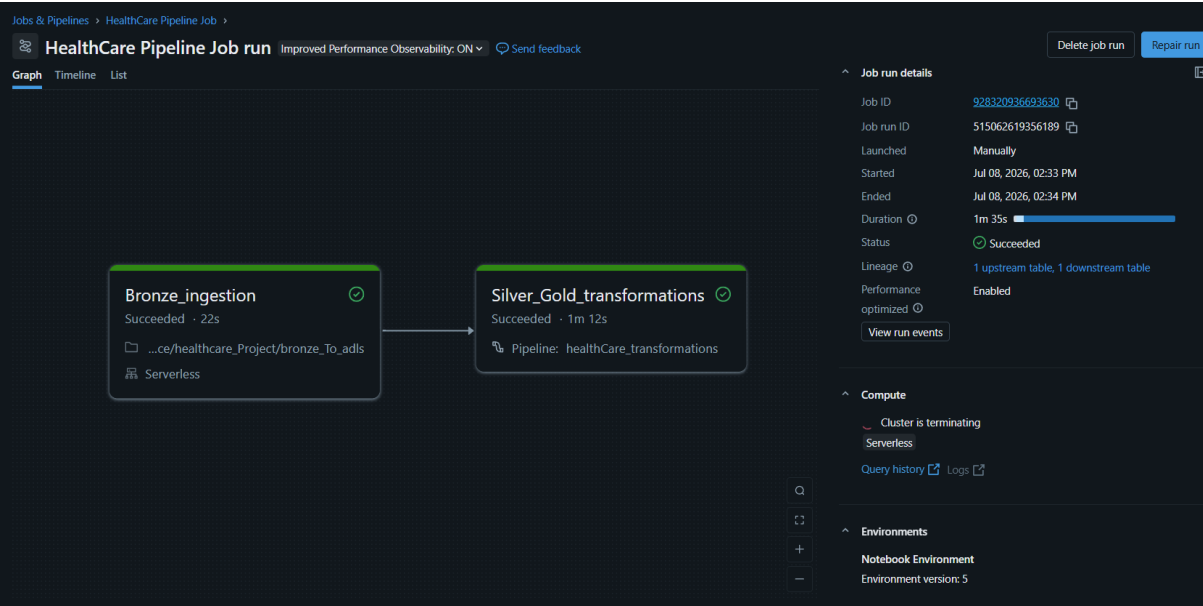
Screenshot 5: Pipeline Dependency Graph



Description:

The pipeline graph illustrates the dependency between the Silver transformation and multiple Gold materialized views, representing the Medallion Architecture workflow.

Screenshot 7: Successful Pipeline Job Execution



Description: The Healthcare Data Pipeline executed successfully, completing Bronze ingestion followed by Silver and Gold transformations through an automated Lakeflow Pipeline.

Screenshot 8: Gold Layer Output Tables

The screenshot shows a data pipeline interface with a dark theme. At the top, a status bar indicates the execution was successful in 1 minute (15s) and shows a 'SQL' button. Below this, a code editor contains the following SQL query:

```
%sql
select * from healthcare.transformations.gold_admission_type_analysis
```

Below the code editor, there is a link to 'See performance (1)' and a 'Checked' status. A message indicates that the result is stored as a Spark DataFrame: `_sqldf: pyspark.sql.connect.dataframe.DataFrame = [admission_type: string, total_admissions: long ... 1 more field]`.

The main part of the screenshot displays a table with the following data:

	admission_type	total_admissions	average_bill
1	Emergency	18070	25551.13
2	Elective	18437	25663.34
3	Urgent	18353	25568.44

Below the table, there is a download icon and a message: '3 rows | 14.608s runtime'. A note at the bottom states: 'This result is stored as _sqldf and can be used in other Python and SQL cells.'

At the bottom of the screenshot, a second code editor is visible, showing the following SQL query:

```
%sql
select * from healthcare.transformations.gold_average_billing_by_hospital limit 5
```

The second code editor also has a 'SQL' button and a '2' in the top right corner.

Description:

The Gold layer contains multiple business-oriented analytical tables, including patient count by hospital, hospital ranking, admission type analysis, medical condition analysis, and average billing by hospital.

Screenshot 9: GitHub Repository Structure

